ÁvilaOS - Resumen de defensa

Elio Colmenares · Proyecto individual · Sistemas Operativos

1. Explicación de apertura

«ÁvilaOS simula varios computadores con CPU, RAM, núcleo y E/S propios. Un reloj global coordina un hilo por computador. La interfaz observa instantáneas y envía órdenes entre ciclos. Podemos cambiar planificación, crear procesos y buffers y estudiar CPU, plazos y sincronización local o remota».

2. Recorrido de un proceso

NEW espera admisión de memoria. READY conserva RAM y espera CPU. RUNNING ejecuta una instrucción, incrementa PC/MAR y contabiliza usuario o núcleo. Puede volver a READY por apropiación o pasar a BLOCKED por E/S, red o semáforo. Al despertar conserva contexto. TERMINATED libera recursos y conserva estadísticas. El deadline avanza incluso sin CPU y puede vencer también en NEW.

3. Arquitectura que debes señalar

Computer ensambla componentes. Kernel admite, despacha, bloquea y termina. Cpu ejecuta; Ram reserva/libera; IoController procesa solicitudes por ciclos. Scheduler utiliza SchedulingPolicy. Simulation aplica comandos y GlobalClock coordina ComputerWorker mediante semáforos Java. UiSession gestiona creación/cierre/archivos fuera del EDT; Swing observa copias inmutables. No hay una clase diferente por computador ni migración de procesos.

4. Las cuatro políticas

• FCFS: orden de entrada a listos; no apropia por la llegada de otro listo.

• EDF: deadline absoluto más próximo; puede apropiar.

• Round Robin: turnos de quantum instrucciones cuando existe competencia.

• Prioridades: menor número = mayor prioridad; puede apropiar.

Empates estables. Cambiar política/quantum se aplica en un límite de ciclo, sin reiniciar PC ni perder RAM. Los milisegundos alteran el ritmo visible, no la duración lógica de las instrucciones.

5. Dos niveles de semáforos

Los Java Semaphore coordinan Threads y acceso real. Los SimulatedSemaphore administran permisos y colas de PCB. Wait puede bloquear el proceso simulado sin detener el reloj ni dejar detenido para siempre el worker de esa máquina.

Por buffer: vacíos, llenos y mutex. Productor: wait(vacíos), wait(mutex), insertar, signal(mutex), signal(llenos). Consumidor: wait(llenos), wait(mutex), extraer, signal(mutex), signal(vacíos). Cada operación es instrucción de núcleo. No repetir un wait al despertar. La red espera antes de retener permisos. Al vencer un plazo, limpiar reservas sin deshacer contenido ya confirmado.

6. Persistencia

El JSON guarda el escenario inicial aplicado, no el estado en mitad de ejecución. Validar todo antes de sustituir el motor; un error conserva el anterior. Guardar temporal completo y después reemplazar. Cargar/reiniciar vuelve a cero y cambia de generación; los borradores y cambios vivos no se cuelan en el escenario guardado.

7. Fórmulas y cuidado al interpretarlas

Throughput = completados / ciclos. CPU = instrucciones / ciclos disponibles; global divide por computadores × ciclos. Respuesta = primera ejecución – llegada, promedio entre quienes ya recibieron CPU. Cumplimiento = completados / (completados + vencidos). Jain = servicio total² / (admitidos × suma de servicios²). Servicio incluye usuario/núcleo; admitidos sin servicio y terminados cuentan, vencidos NEW no.

Sin denominador: Sin datos. No sumar porcentajes de máquinas. Espera de semáforo = tiempo de episodios cerrados / episodios cerrados; abiertos aparte. No incluir E/S ni red. Una muestra del gráfico por ciclo completo, aunque el Timer no muestre todos los refrescos.

8. Demo breve y cálculo manual

Cargar examples/local-producer-consumer.json; dar doce pasos. C y P terminan: servicio usuario 2, núcleo 10, CPU global 50 %, throughput 2/12=0.167, respuesta 0.5, cumplimiento 100 % y Jain 1. B1 produjo/consumió 1/1, ocupación 0, vacíos/llenos/mutex 1/0/1. Una espera cerrada de 6 ciclos da promedio 6.

Reiniciar: cero y métricas vacías. Cargar remote-producer.json; tras ocho pasos PC=1,1,1,2,3,4,5,6, CPU global37.5 %, producido1. Los pasos de red no avanzan PC. Guardar/recargar y archivo corrupto deben respetar el escenario y mostrar errores sin perder ejecución.

9. Preguntas probables

¿Por qué hilos y reloj? Un hilo por computador realiza su trabajo y confirma; el coordinador mantiene el límite global y el orden reproducible. No son temporizadores independientes que puedan desfasarse.

¿Bloqueado libera RAM? No. Conserva reserva y contexto; la terminación sí libera y cancela recursos pendientes.

¿Puede vencer en NEW o esperando mutex? Sí. Se limpia la espera/reserva y se registra DEADLINE_EXPIRED; no se inventan instrucciones de servicio.

¿Por qué FCFS tiene respuesta 0 en la comparación? Solo respondieron 2 de 4; otros vencieron sin CPU. Ese promedio no prueba que atendió mejor a toda la carga.

¿EDF y prioridades siempre ganan? No. Coinciden en la carga medida porque sus órdenes coinciden. RR con quantum2 y plazos cortos venció más procesos; otras cargas/quantums podrían cambiarlo.

¿La red siempre añade dos ciclos al tiempo total? Al productor aislado sí en la prueba L2:6 -> 8. En pareja C/P:12 -> 13 porque P empieza antes en otra CPU. Distinguir latencia, solapamiento y horizonte de medición.

10. Antes de defender

Saber localizar las clases y las dos UML; explicar una traza sin memorizar solo cifras. Compilar/ejecutar en NetBeans, practicar pausa/paso/reinicio/PCB y completar checklist de archivos, cierre, Debug y copia limpia. Comprobar acceso de correctores a GitHub y enviar el informe por el canal académico. Consultar las guías por módulo para ampliar; este resumen es el recorrido esencial.